



GPH 二进制格式描述（逆向工程）

1. 概述

GPH 是一种几何/多面体网格文件格式，可能来自 SCTpre 或 CGNS 相关 CFD 工具。

- **魔数标识:** CRDL-FLD（8 字节 ASCII）
- **字节序:** 大端序（Big-Endian），所有多字节整数和浮点数
- **对齐:** 4 字节对齐

2. 文件布局

偏移 (hex)	大小	节名称	描述
0x0000-0x001C	28 B	文件头	CRDL-FLD 标识 + 维度
0x001C-0x0078	92 B	FileRevision	I4 标量（如 2025）
0x0078-0x00D8	96 B	Application	C1[8], 如 "SCTpre"
0x00D8-0x0134	92 B	ApplicationVersion	I4
0x0134-0x0190	92 B	ReleaseDate	日期字符串
0x0190-0x01EC	92 B	GridType	字符串
0x01EC-0x0248	92 B	Dimension	I4（空间维度）
0x0248-0x02A4	92 B	Bias	I4

偏移 (hex)	大小	节名称	描述
0x02A4-0x0300	92 B	Date	日期字符串
0x0300-0x03A8	168 B	Comments	注释字符串
0x03A8-0x04E0	312 B	Cycle	I4 + 单位信息
0x04E0-0x0560	128 B	Unused	保留
0x0560-0x05D8	120 B	Encoding	如 "UTF-8"
0x05D8-0x0600	40 B	HeaderDataEnd	头部结束标记
0x0600-0x0628	40 B	OverlapStart_0	重叠区开始
0x0628-0x08DC	692 B	LS_CvolIdOfElements	I4[n_cells] 每单元 cvol_id (见 §5.1)
0x08DC-0x26B0	变长	LS_Links	面拓扑 (owner/neighbor/npe/conn, §5.2)
0x26B0-0x2C60	变长	LS_Nodes	R8[n,3] 顶点坐标 (三轴 float64 块, §5.3)
...	变长	LS_SurfaceRegions	命名边界面区域 (§5.4)
...	变长	LS_SolverUnusedRegions	求解器内部区域名 (可选)
...	变长	LS_VolumeRegions	体区域名 → CGNS Zone (§5.5)
...	变长	LS_Parts	Part 名 + cvol_id 描述符 (§5.6)

偏移 (hex)	大小	节名称	描述
...	变长	LS_Assemblies	XML 装配树 → Zone 命名 (§5.7)
0x42B0-0x45A4	756 B	Element_InformationFlag	单元标志
0x45A4-EOF	-	OverlapEnd	文件尾

注意：上表偏移仅适用于 `box.gph` 等早期样例；`tr03.gph`、`laptop_*.gph` 等文件体积更大，须用 `gph_parser.py` 或 `gph_model.find_section()` **动态定位**各节，不可硬编码偏移。

3. 记录格式

每个命名字段遵循以下模式：

```
[长度 I4] = 0x20 (32)
[名称 C1[32]] = 32 字节 ASCII，空格填充
[长度 I4] = 0x20 (32)    ; 可选，节标记
[描述符 变长]
- 0x0C 0x04 [dim0] [dim1] : 维度，类型 04 = I4
- 0x0C 0x08 [dim0] [dim1] : 类型 08 = R4 或 I8
[值 / 数组数据]
```

数据类型代码（推断）

代码	类型	说明
0x04	I4	32 位有符号整数
0x08	R4 / I8	32 位浮点或 64 位整数（视上下文）

4. 元数据字段

字段	类型	示例
FileRevision	I4	2025
Application	C1[8]	"SCTpre"
ApplicationVersion	I4	1
ReleaseDate	string	日期
GridType	string	网格类型
Dimension	I4	3
Bias	I4	-
Date	string	-
Comments	string	-
Cycle	I4	仿真步
Encoding	C1	"UTF-8"

5. 数据数组

数组名	类型	说明
LS_CvolIdOfElements	I4[n_cells]	每单元的 cvol_id （与 LS_Parts 描述符对应，§5.1）
LS_Links	I4[]	面拓扑（owner / neighbor / npe / [face_type] / conn，§5.2）
LS_Nodes	R8[n,3]	顶点坐标（三轴 float64 块，§5.3）
LS_SurfaceRegions	变长	命名边界面区域 → CGNS ZoneBC（§5.4）
LS_SolverUnusedRegions	变长	求解器内部区域名（可选）

数组名	类型	说明
LS_VolumeRegions	变长	体区域名列表 → CGNS Zone (§5.5)
LS_Parts	变长	Part 定义 (名 + cvol_id, §5.6)
LS_Assemblies	XML	装配层次 → Zone 命名规则 (§5.7)
Element_InformationFlag	变长	单元信息标志

5.1 LS_CvolIdOfElements

每个体单元一条 I4，值为该单元所属 Part 的 **cvol_id** (scFLOW 内部分区标识)。

- **不是** LS_Parts 列表中的 1-based 序号。
- tr03.gph：63 882 单元，cvol_id ∈ {1, 2} 对应 Case / Rotate。
- laptop_simplified_voxel_less.gph：cvol_id ∈ {1, 9, 11}，与 Part 列表顺序无关。

gph2cgns.py 用 LS_Parts 中每个 Part 名块后的 [12, 4, cvol_id, 4] 描述符建立 part_name → cvol_id 映射，再与 LS_CvolIdOfElements 匹配生成各 Zone 的 cell 子集。

5.2 LS_Links (含多面体网格)

按数据块顺序 (经通用块扫描器定位)：

块	大小	含义
owner	n_faces × I4	面的 owner 单元 (0-indexed)
neighbor	n_faces × I4	对侧单元；0xFFFFFFFF = 边界面
npe	n_faces × I4	每面节点数 (纯三角时全为 3；tr03 为 3..11)
face_type	1 × I4	仅部分旧文件 ：单元类型标志
conn	sum(npe) × I4	0-based 顶点索引， CSR 布局

CSR 索引：face_offsets[0]=0，face_offsets[i+1]=face_offsets[i]+npe[i]；面 i 的节点为 conn[face_offsets[i]:face_offsets[i+1]]。

纯三角 legacy 文件可能使用列主序 conn；gph2cgns 根据块大小与 sum(npe) 自动判别。

5.3 LS_Nodes

节点坐标按 X / Y / Z 三个等长 float64 轴块存储（经块扫描取三个最大且等长的 f64 块）。

实际编码（统一格式）

```
[16B 描述符] 00 00 00 0C / 00 00 00 08 / n_verts / 00 00 00 01
[8B 块头] 00 00 00 0C / byte_count
[n_verts × 8B] 标准大端 IEEE-754 float64
[4B 尾部] byte_count（哨兵，等于块字节数）
```

- 轴顺序：**X, Y, Z**（与 CGNS 一致，无需重排）
- 每块末尾有 4 字节的 `byte_count` 哨兵

示例（来自 `box_ansa.gph`）：坐标值 `float32(0.01)` 加宽为 float64 = `0x3F847AE140000000`，在文件中按标准大端写为 `3F 84 7A E1 40 00 00 00`。

示例（来自旧 `box.gph`）：坐标值 0.01（精确）= `0x3F847AE147AE147B`，在文件中按标准大端写为 `3F 84 7A E1 47 AE 14 7B`。

历史"字反转 float64"误读

原始 `gph2cgns.py` 跳过了 12 字节块头（而正确应该跳过 8 字节），并使用字反转读法。对**特定**字节序列，这两个错误恰好相互抵消，使少数顶点产生看似合理的数值——但大多数顶点会得到 `1e37` 等夸张幅值。当前实现已修正为正确的"跳 8 字节 + 标准大端"读法。

防御性方言自检

`gph2cgns.py` / `gph_model.py` / `gph_parser.py` 扫描节内三个等长 float64 块，对标准大端与字反转两种读法打分，选取物理合理者；字反转选中时对磁盘轴序 X,Z,Y 做 Y/Z 交换。

5.4 LS_SurfaceRegions

每个表面区域由 **三个连续数据块** 组成：

1. **name** — ASCII, NUL/空格填充
2. **face_ids** — `l4[m]`, 全局面索引 (0-based, 对应 `LS_Links` 面数组下标)
3. **weights** — `l4[m]`, 与 `face_ids` 等长（目前观测值均为 1）

`gph2cgns.py` 为每个 Zone 投影这些区域，在 `ZoneBC` 下为每个区域名写一个 `BC_t` (`PointList` 为 zone 内 1-based 面号；该 zone 无此区域面时省略 `data` 数据集，与 `tr03_orig.cgns` 一致)。

5.5 LS_VolumeRegions

ASCII 字符串块列表，顺序即 CGNS 中体区域 Zone 的生成顺序（如 `FluidRegion`、`Rotate_MovingVolumeRegion` 等）。`gph2cgns` 按名称与 `LS_CvolIdOfElements` / `Part` 启发式匹配 cell 子集。

5.6 LS_Parts

每个 Part 记录布局（节选）：

```
[12,4,1,1] ... [12,4,255,4] [12,1,255,1]
[<part name>, 255 B ASCII]
[trailer=255]
... 若干描述符 ...
[12, 4, <cvol_id>, 4] ← 该 Part 的 cvol_id（在下一 Part 名块之前）
```

Zone 命名规则（与 FLDUTIL 对齐，见 `DEV_SUMMARY.md` §10）：

- 路径深度 ≥ 2 （含多个 `.`）→ 直接用完整路径（如 `laptop_3d_geom.____.air_domain`）
- 否则加 `FPHPARTS.` 前缀（如 `FPHPARTS.tr03.Case`）
- 根级 Part + 空 assembly 前缀 → `fan2.fan1.rotation1` 等

5.7 LS_Assemblies

UTF-8 XML，描述 `<assembly>` / `<part>` 层次。`gph2cgns` 解析 `part_paths` 与 `root_empty_prefix`（首个顶层 assembly 下、与根级 part 数量相同的空 assembly 名拼接），用于 Part Zone 命名。

6. CGNS 多 Zone 输出概要

`gph2cgns.py` 根据 `LS_VolumeRegions` + `LS_Parts` + `LS_Assemblies` 生成多个 `Zone_t`，每个 Zone 为全局网格的子集（顶点/面重编号），并附带：

- `GridElements_Faces` (`NGON_n=22`)
- 与 `Zone` 同名的 `NFACE_n` (带符号面索引)
- `ZoneBC` : 来自 `LS_SurfaceRegions` 的命名 BC 族
- `FlowSolution` 占位

无分区元数据时回退为 `FluidRegion` + `FPHPARTS.box_vol` 两 `Zone` (`box_ansa.gph` 行为)。

7. 使用 Python 解析

运行：

```
python gph_parser.py [box.gph]
```

将输出节布局、数据采样和完整格式说明。

8. 参考

- 与 CGNS 的 ADF (Advanced Data Format) 相似
- 32 字符标签符合 ADF 节点标签约定
- CRDL-FLD 可能表示 "Card/Record Field" 或厂商自定义格式

9. CGNS 输出 HDF5 格式约束 (重要)

本节描述的不是 GPH 输入格式，而是 `gph2cgns.py` 写出的 **CGNS 输出文件**在底层 HDF5 层面必须满足的兼容性约束。详细推导见 `DEV_SUMMARY.md` 第 9 章。

9.1 HDF5 superblock 版本对下游 CFD 工具的影响

HDF5 文件起始 8 字节是固定魔数 `\x89HDF\r\n\x1a\n`，紧接其后的第 9 字节 (offset 8) 是 **superblock 版本号**。常见的三种版本对应不同的 HDF5 文件格式特性：

superblock 版本	触发条件	object header 格式	组存储	典型读
v0	<code>libver_low='earliest'</code> (h5py 默认)	v1 (无 4 字节签名)	v1.6 sym-table (SNOD + TREE + HEAP)	所有 CGNS 工具: ANSA、Tecplot 官方库
v1	非默认 <code>H5Pset_sym_k</code> 或 <code>H5Pset_istore_k</code>	v1	sym-table, 但 B-tree 参数可调	同 v0, 绝大多数工具
v2	<code>libver_low='v108'</code> 或更高	v2 (带 OHDR 4 字节签名)	compact link 或 dense (fractal heap)	⚠ 部分老旧 CGNS ANSA 会报 <code>No bas</code>

9.2 已知不兼容案例：ANSA "No bases found!"

症状：用 ANSA 导入 `gph2cgns.py` 生成的 CGNS 文件时报错：

```
No bases found!
```

根因：之前的提交曾设置 `h5py.File(libver=("v108", "v108"))` 以启用 HDF5 1.8 紧凑链接存储 (compact link storage)，把输出文件从 73 KB 缩到 31 KB。该设置生成 **v2 superblock**，而 ANSA 的 CGNS 读取器只接受 v0 / v1 superblock，遇到 v2 时直接判定文件无 Base 节点。

修复：固定使用 `libver=('earliest', 'v108')` (h5py 默认)，即 **v0 superblock + v1.6 sym-table 布局**。代价是文件比官网 vendor 参考大约 2 倍 (73 KB vs 35 KB)，但保证所有目标 CFD 工具都能读取。

9.3 vendor (`box_ansa_orig.cgns`) 的特殊紧凑布局

官网 `FLDUTIL` 工具产生的参考文件 `box_ansa_orig.cgns` 兼具两者之长：

- superblock **v0** (ANSA 可读)

- 但**无** SNOD / TREE / HEAP (即不是 v1.6 sym-table)
- 也**无** OHDR 签名 (即不是 v2 object header)

逐字节解码根组 object header 可见使用了 **v1 object header 内嵌 HDF5 1.8 Link Info / Group Info 消息**:

```
Object header @ 0x60 (root group):
  version: 1                                ← v1 object header (无签名)
  msg 0: type=0x02 (Link Info), size=40     ← HDF5 1.8 紧凑链接
  msg 1: type=0x0A (Group Info), size=8     ← HDF5 1.8 组信息
  msg 6: type=0x0C (Attribute), size=40
  ...
```

该布局当前无法通过 h5py 复刻:

- h5py.h5p.PropGCID 未导出 set_link_phase_change Python 包装。
- 即使通过 ctypes 直接调用 C 层 H5Pset_link_phase_change 在 GCPL / FCPL 上设置成功, HDF5 在 libver_low='earliest' 时仍会回退到 v1.6 sym-table 存储, 无法触发 1.8 紧凑链接消息。

因此 gph2cgns.py 选择最稳妥的 **v0 superblock + v1.6 sym-table** (73 KB), 与 vendor 内容字段级 PERFECT MATCH, 仅 HDF5 元数据布局不同。

9.4 HDF5 签名速查表 (用于逐字节诊断 CGNS 输出)

签名 (4 字节 ASCII)	含义	出现意味着
\x89HDF\r\n\x1a\n	HDF5 魔数 (offset 0)	文件是 HDF5 容器
OHDR	v2 object header	superblock 必为 v2, ANSA 等老读取器可能拒绝
OCHK	object header continuation chunk (v2)	同上
SNOD	v1.6 symbol-table node	v0/v1 superblock + v1.6 组存储
TREE	v1 B-tree	同上, 每组一个

签名 (4 字节 ASCII)	含义	出现意味着
HEAP	v1 local heap	同上, 每组一个
FRHP	fractal heap header	dense link 或 attribute 存储, HDF5 1.8+
BTHD	v2 B-tree header	dense link 索引, HDF5 1.8+
GCOL	global heap collection	变长数据集 (如 string) 共享存储

诊断命令示例:

```
python3 -c "
data = open('your.cgns','rb').read()
print(f'size={len(data)} sb_v={data[8]}')
for sig in (b'OHDR', b'SNOD', b'TREE', b'HEAP', b'FRHP', b'BTHD'):
    print(f' {sig.decode()}: {data.count(sig)}')
"
```

9.5 输出文件 sanity check

gph2cgns.py 生成的 CGNS 应满足:

- data[8] == 0 (superblock v0)
- OHDR count = 0
- SNOD count > 0 (与组数量相当)
- 通过 h5py.File(path, 'r') 可正常打开并读取 Base / Base/<Zone> / 各 Elements_t 子节点
- ZoneType 数据集字节序列为 Unstructured